

Noun / Verb / TTC Pipeline — From Ego4D Annotations to Final Evaluation

This document explains, end-to-end, how the code in `Thesis_main/` and `local_extraction/` uses **nouns, verbs, TTC, and related elements** to go from raw Ego4D STA annotations to the final evaluation metrics (accuracy, mAP, TTC MAE) reported for Tracks A, B, and C.

The description is grounded in the actual scripts:

- Label builder: `local_extraction/labels/sta_builder_local.py`
 - Track A: `local_extraction/trackA/trackA_stageA/trackA_stageA.py`,
`.../trackA_stageB/trackA_stageB.py`
 - Track B: `local_extraction/trackB/*.py`
 - Track C: `local_extraction/trackC/trackC_pruning.py`
 - Planning / decisions: `Thesis_main/thesis_plan_tracks_ABC_v2.md`,
`Thesis_main/DECISION_noun_class_alignment.md`
-

1. Semantic Ground Truth in Ego4D STA

Ego4D STA provides, for each annotated **short-term interaction**, object-centric labels:

- `box: [x1, y1, x2, y2]` — next-active object location (in decision frame).
- `verb_category_id` — verb class index.
- `noun_category_id` — noun class index.
- `time_to_contact` — TTC in seconds until interaction onset.
- `video_uid`, `clip_uid`, `frame`, `clip_frame`, and other clip metadata.

The structure is documented in `ego4d_samples/_auto_fields.md` and the dataset README, and materialized locally through the FHO STA JSONs:

- `fho_sta_train.json`, `fho_sta_val.json`
- or their height-540 variants: `fho_sta_train_height-540.json`, `fho_sta_val_height-540.json`

All downstream use of **noun / verb / TTC** in this repo ultimately traces back to these fields.

2. Local Label Builder — Turning Ego4D STA into Local Manifests

File: `local_extraction/labels/sta_builder_local.py`

This script is the bridge between the official Ego4D annotations and our **local_extraction** workspace. It:

1. **Loads STA objects** from FHO JSONs.
2. **Emits YOLO labels** (optional) so Stage A can run on local frames.
3. **Builds “head manifests”** that attach (`verb_id`, `noun_id`, `ttc`) to each decision frame and GT box.

2.1 StaRec: unified representation of objects

The core dataclass (simplified) is:

- `video_uid, clip_uid`
- `video_frame, clip_frame`
- `box: [x1,y1,x2,y2]`
- `verb_id: Optional[int]`
- `noun_id: Optional[int]`
- `ttc: Optional[float]`
- `split: "train" | "val"`
- `width, height`

`_load_one_file()` iterates over `rec["objects"]` in each annotation record and fills a `StaRec`:

- `verb_id` from `obj["verb_category_id"]` or `obj["verb_id"]`.
- `noun_id` from `obj["noun_category_id"]` or `obj["noun_id"]`.
- `ttc` from `obj["time_to_contact"]` or `obj["ttc"]`.

So **all three semantic elements** are captured per object.

2.2 Optional YOLO labels — using noun IDs

If `GENERATE_YOLO_LABELS=True`, the script normalizes boxes and writes YOLO txt files under:

- `local_extraction/<VERSION>/yolo_labels_540/{videos,clips}/<uid>/<frame:07d>.txt`

For each `StaRec` in an image:

- It computes `(cx, cy, w, h)` normalized by image width/height.
- The class ID is derived from the noun:
 - If `DETECTOR_CLASS_POLICY == "single"` → `cls_id = 0`.
 - Else → `cls_id = noun_id` (or 0 if missing).

These labels are used in two ways:

- To **train a YOLO detector** (e.g., a single-class “next_active” detector) if desired.
- To evaluate Stage A proposals in **oracle mode** (`trackA_stageA.py` / `trackA_stageB.py`) by comparing proposals against these YOLO labels.

Decision-wise, `DECISION_noun_class_alignment.md` records that for the current milestone we intentionally **do not** fine-tune YOLO on the full noun taxonomy and often operate in **single-class mode**. Nouns are still used to construct these labels but Stage A is treated as **noun-agnostic**, focusing on **recall@K**.

2.3 Head manifests — selecting one GT object per image

The same builder optionally emits **head manifests** under:

- `local_extraction/<VERSION>/manifests/{head_train,head_val}_{video,clip}.json`

Mechanism:

1. Group `StaRec` entries by `(video_uid, video_frame, split)` (and analogously for clips).

2. For each group, either:

- Choose **one object with minimum TTC** (if `SELECT_MIN_TTC_FOR_HEAD=True`), or
- Keep all objects (if disabled).

3. Write a list of dicts, each with:

```
{
  "image": "<frames_root>/<uid>/<frame:07d>.jpg",
  "gt_box": [x1, y1, x2, y2],
  "verb_id": <int or 0>,
  "noun_id": <int or 0>,
  "ttc": <float or 0.0>
}
```

This is the first place where we **collapse potentially multiple objects in an image** down to the one we care about for STA: the object with the **earliest upcoming interaction** (min TTC). Its `verb_id`, `noun_id`, and `ttc` become the canonical **per-image ground truth** that Stage B and Track B build on.

3. Track A — From Frames and YOLO Labels to Candidate Boxes

Track A is the **proposal stage**: it turns decision-frame images into **top-K candidate boxes** that may or may not match the true next-active object.

3.1 Stage A: YOLO proposals on the last frame

File: `local_extraction/trackA/trackA_stageA/trackA_stageA.py`

Inputs:

- Frames: `local_extraction/v2/extracted_frames/<uid>/<frame:07d>.jpg`
- Optional YOLO labels (from the label builder):
`local_extraction/v2/yolo_labels_540/{clips|videos}/<uid>/<frame:07d>.txt`

Key toggles:

- `DETECTION_MODE = "yolo" or "oracle"`
 - `"yolo"`: run Ultralytics YOLO (weights usually trained as **single-class "next_active"**).
 - `"oracle"`: replay YOLO txt labels as proposals.
- `K`: top-K candidates kept per image.
- `LAST_FRAME_ONLY=True`: pick the decision frame per UID.

For each image:

1. Run YOLO or read the YOLO txt (`oracle_boxes()`).
2. Yield boxes as `{x1,y1,x2,y2,conf,cls}`.
3. Keep top-K by `conf`.

Outputs:

- `runs/Track_A/StageA_*/candidates.jsonl`
 - One line per image: `{"uid": uid, "frame": frame_idx, "boxes": [...]}`.
- `summary.json` with counts and YOLO config.

The **class field `c1s`** is either:

- A **noun-derived class index** if YOLO was trained with noun IDs, or
- A **single “next_active” class** (when using single-class YOLO).

Per `DECISION_noun_class_alignment.md`, the current pipeline **treats these proposals as noun-agnostic**: the **head model** is responsible for semantics; Stage A’s main role is to achieve high proposal **recall@K**.

3.2 Stage B: Cropping candidates and attaching STA semantics

File: `local_extraction/trackA/trackA_stageB/trackA_stageB.py`

Inputs:

- `candidates.jsonl` from Stage A.
- Original frames in `v2/extracted_frames`.
- Optional YOLO txt labels for recall evaluation (`EVAL_WITH_LABELS=True`).
- **Head manifests** from the label builder:
`local_extraction/v2/manifests/head_{train,val}_{clip,video}.json`.

Main responsibilities:

1. Crop ROIs for each candidate:

- Clamp box to image size.
- Crop from the RGB frame.
- Optionally resize to `CROP_SIZE` (e.g., 256×256).
- Save under `runs/Track_A/trackA_stageB_*/crops/...`

2. Evaluate proposal quality (recall@K):

- Read GT boxes from YOLO txt (`read_yolo_labels()`).
- For each candidate ROI, compute IoU with each GT box.
- Mark whether any candidate hits $\text{IoU} \geq \text{IOU_THRESH}$ (0.5).
- Aggregate:
 - `images_with_gt`
 - `images_with_hit`
 - `recall_at_K`
 - `mean_best_iou`
 - IoU histogram.
- Write metrics into `StageB summary.json` and optionally back into Stage A’s `summary.json`.

3. Merge candidates with head manifests (verb/noun/TTC)

This is where **Ego4D semantics** (verb, noun, TTC) are aligned with candidate boxes.

- `load_head_manifest(path)` loads the label-builder’s head JSON into a map:

- Key: `(uid, frame)`
- Value: `{gt_box, verb_id, noun_id, ttc, image}`
- For each candidate crop `(uid, frame, candidate_box)`:
 1. Look up the head manifest entry for `(uid, frame)`.
 2. Compute IoU between `candidate_box` and `gt_box`.
 3. Define `is_positive = (IoU >= IOU_THRESH)`.
 4. Record:

```
{
  "image_path": "<full path to frame>",
  "candidate_box": [cx1, cy1, cx2, cy2],
  "candidate_conf": conf,
  "candidate_cls": cls,
  "verb_id": <verb_id>,
  "noun_id": <noun_id>,
  "ttc": <ttc>,
  "is_positive": true|false,
  "iou": <IoU value>
}
```

- These per-candidate records are written to:
 - `runs/Track_A/trackA_stageB_*/head_train.jsonl`
 - `runs/Track_A/trackA_stageB_*/head_val.jsonl`

Additionally, `manifest.jsonl` and `manifest.csv` capture crop-level metadata, optionally including the same `verb_id`, `noun_id`, `ttc`, `is_positive`, and `iou` fields.

Result: Stage B converts the **per-image STa semantics** (from head manifests) into **per-candidate training signals**:

- A **binary label** `is_positive` telling whether a candidate box matches the min-TTC GT object.
- A **scalar TTC target** associated with each candidate.
- **Verb and noun IDs** attached as metadata for possible future multi-task heads.

4. Track B — Dataset, Fusion Head, and Training/Eval Metrics

Track B takes the **Stage B head manifests** and trains a **lightweight fusion head** that predicts:

- For each candidate: a **class label** (currently "next_active" vs not).
- For each candidate: a **TTC value** (in seconds).

4.1 TrackBDataset: parsing manifests into training samples

File: `local_extraction/trackB/trackB_dataset.py`

Construction:

- `TrackBDataset(frames_root, manifests_root, manifest_path, ...)`
 - `frames_root` = `local_extraction/v2/extracted_frames`
 - `manifests_root` / `manifest_path` typically point to a Stage B run:
 - `runs/Track_A/trackA_stageB_*/head_train.jsonl`
 - `runs/Track_A/trackA_stageB_*/head_val.jsonl`

Manifest discovery:

- `latest_stageB_run()` finds the newest Stage B directory with a head manifest.
- `discover_stageB_manifest()` and `resolve_stageB_manifest()` locate `head_train` / `head_val` inside it.

4.1.1 Converting Stage B records

Stage B records are converted by `_maybe_convert_stageB_records()`:

- Inputs: each dict from `head_train.jsonl` / `head_val.jsonl` including:
 - `image_path`, `candidate_box`, `is_positive`, `ttc`, etc.
- Grouped by `(uid, frame_name)` where:
 - `uid` is derived from the parent folder of `image_path`.
 - `frame_name` is the image filename (e.g., `0001234.jpg`).
- For each candidate:
 - Classification label `cls` is derived from `is_positive`:
 - `cls` = 1 if `is_positive` is truthy.
 - `cls` = 0 otherwise.
 - TTC `ttc` is `_safe_float(rec["ttc"])`.
- For each `(uid, frame_name)` group, we build:

```
{
  "uid": uid,
  "frame_name": frame_name,
  "frame_idx": <optional>,
  "image": "<absolute path>",
  "candidates": [
    {"x1": ..., "y1": ..., "x2": ..., "y2": ..., "cls": 0/1, "ttc": float},
    ...
  ]
}
```

Important: here we **discard verb_id and noun_id** for the current Track B setup and keep:

- `cls` = "is this candidate the min-TTC next-active object?" (binary).

- `ttc` = time-to-contact for that object.

The schema and metrics code still support **multi-class classification** (e.g., to predict noun or verb classes later), but the installed pipeline uses binary `cls` at this phase.

4.1.2 Parsing generic manifests

`_parse_record()` also supports more generic manifests (e.g., from notebooks or future experiments) by:

- Accepting candidate lists under keys `candidates` / `boxes` / `objects`.
- Mapping box coordinates from `{x1, y1, x2, y2}` or synonyms (`xmin, xmax, left, right`, etc.).
- Inferring `cls` from:
 - `c["cls"]`, or
 - `c["label"]` with mapping of booleans / strings to `{0,1}`.
- Reading `ttc` from `c["ttc"]` or `c["time_to_contact"]`.

If no candidate list exists but `gt_box` is present, it fabricates a single candidate with:

- `bbox = gt_box`
- `cls = verb_id` if present, otherwise `noun_id`, otherwise `0`
- `ttc = r["ttc"]`

This means the same dataset machinery could be reused to train **noun or verb heads** by changing manifest content, even though the current Stage B pipeline uses binary `is_positive`.

4.1.3 TTC statistics and normalization

`_compute_ttc_stats()` aggregates TTC over all candidates:

- `ttc_min, ttc_max, ttc_mean, ttc_std, total_records, total_candidates`.

If `normalize_ttc=True` (the default in `TrainConfig`), each candidate gets:

- `ttc_norm = (ttc - mean) / max(std, 1e-6)`.

`__getitem__()` then yields, per sample:

- `bboxes`: list of candidate boxes.
- `labels`: tensor of `cls` labels.
- `ttc`: tensor of raw TTC values.
- `ttc_norm`: tensor of normalized TTCs.
- `img_tokens, vid_tokens, hw, image_size`, etc. for fusion.

4.2 Fusion and head: predicting cls + TTC per candidate

Fusion: `local_extraction/trackB/trackB_fusion.py`

- `TrackBFusion(FusionConfig)` implements:
 - **Frame-Guided Temporal Pooling (FGTP)**: pools video tokens over time into last-frame spatial grid.
 - **Dual cross-attention** between image and pooled video tokens.

- Outputs:
 - `fused_img`: (B, N, C)
 - `fused_vid`: (B, N, C)

Head: `local_extraction/trackB/trackB_head.py`

- `TrackBHead(HeadConfig)` takes `fused_img` and `fused_vid`:
 - Concatenates along channel: `[fused_img || fused_vid]`.
 - Passes through a small MLP.
 - Outputs:
 - `cls_logits`: (B, N, num_classes)
 - `ttc`: (B, N, 1) (regressed TTC in normalized units if training with norm).

By default, `num_classes=2` (binary **next-active vs not**), but checkpoints can encode a different number; `trackB_eval.py` infers `num_classes` from the checkpoint's head weights.

4.3 Training loop: loss functions and how TTC is used

File: `local_extraction/trackB/trackB_train_loader.py`

Key pieces:

- `TrainConfig` holds hyperparameters:
 - `candidate_limit`, `normalize_ttc`, `batch_size`, `epochs`, etc.
- A **DataLoader** is built over `TrackBDataset`, with `trackB_collate` to keep a batch of variable-size candidate sets.

For each batch (for "loader" variant):

1. For each sample:

- Project `img_tokens` and `vid_tokens` via a 512→256 linear `projector`.
- Apply `TrackBFusion` to obtain fused tokens.
- For each candidate box:
 - Use `roi_pool_tokens_mean()` to pool fused image and video tokens inside the box (using `hw` and `image_size`).
- Collect per-candidate features across the batch.

2. Pad and stack candidate features:

- `img_pad`, `vid_pad`: (B, `Nc_max`, C)
- `mask`: (B, `Nc_max`) indicating valid candidates.

3. Forward through `TrackBHead`:

- `out["cls_logits"]`: per-candidate logits.
- `out["ttc"]`: per-candidate TTC predictions.

4. Build targets:

- Classification: `labels_pad` from each sample's `labels` (0/1).
- TTC: `ttc_pad` from each sample's `ttc_norm` (if available) or `ttc`.

5. Loss:

- Classification loss:

```
ce_loss = CrossEntropy(out.cls_logits, labels_pad)
```

- TTC regression loss (per candidate, masked):

```
ttc_loss_all = L1(out.ttc, ttc_pad) # (B,Nc)
ttc_loss = (ttc_loss_all * mask).sum() / mask.sum().clamp_min(1.0)
```

- Combined:

```
loss = ce_loss + TTC_LOSS_WEIGHT * ttc_loss
```

with `TTC_LOSS_WEIGHT = 0.1` by default.

So **TTC is a first-class regression target**, jointly optimized with classification. Nouns and verbs are **not** currently used in the loss; they remain encoded in Stage B manifests for potential extension to multi-task heads (e.g., predicting `(next_active, verb, noun, TTC)`).

4.4 Validation and final evaluation metrics (Track B)

There are two evaluation paths:

1. **Inline validation** in `trackB_train_loader.py` via `_evaluate_loader()`.
2. **Standalone evaluation** in `trackB_eval.py`.

Both share the same metric logic from `trackB_metrics.py`:

- **Classification accuracy:**
 - `binary_accuracy()` for $K=2$
 - `multiclass_accuracy()` for $K>2$
- **Average precision & mAP:**
 - `binary_average_precision()` → AP for positive class (used as mAP in binary case).
 - `per_class_ap()` → AP per class, averaged to mAP for multi-class.
- **TTC error:**
 - Predictions are denormalized:

```
ttc_s = denorm_ttc(ttc_norm, mean, std)
```

- Then TTC MAE is:

```
ttc_mae_seconds = mean(|pred_ttc_s - gt_ttc_s|)
```

Standalone evaluation (`trackB_eval.py`) additionally:

- Saves candidate-level predictions (CSV + JSONL).
- Draws qualitative overlays:
 - For each validation frame, highlighting top-K candidates by predicted `p(next_active)` and annotating predicted vs GT TTC.

Final metrics for Track B runs are stored under:

- `local_extraction/runs/Track_B/metrics/metrics_val_<timestamp>.json`

Each JSON includes:

- `accuracy`
- `mAP`
- `ap_per_class`
- `ttc_mae_seconds`
- `num_candidates`
- checkpoint path and manifest path.

These are the **final evaluation numbers** used for Track B in the thesis procedure.

5. Track C — RGTP Pruning with the Same Semantics

Track C (pruning) reuses the **trained Track B model** and the same **val manifest** but inserts a training-free **Rollout-Guided Token Pruning (RGTP)** step before the head.

File: `local_extraction/trackC/trackC_pruning.py`

Key ideas:

- Use FGTP attention and motion to estimate **per-spatial token importance**.
- For each candidate box, pool importance into a **candidate score**.
- Keep only the top fraction of candidates (by score); others are effectively **masked out**.

5.1 Candidate scoring

- `_grid_rollout()`:
 - Recomputes FGTP attention over time to obtain rollout scores per grid token.
- `_motion_energy()`:
 - Uses differences between last frames to measure motion energy per grid location.
- `_candidate_scores()`:
 - Normalizes rollout and motion to `[0,1]`.
 - Combines them:

```
grid_score = temporal_decay * rollout + (1 - temporal_decay) * motion
```

- Uses `roi_pool_tokens_mean()` to pool grid scores inside each candidate box.
- Returns a scalar importance for each candidate.

5.2 Pruning and evaluation

- `_build_mask(scores, rgtp_cfg)`:
 - Chooses how many candidates to keep based on `rate` and `min_keep`.
 - Builds a boolean mask over candidates.
- Evaluation loop:
 - If all candidates are kept, calls `head` normally.
 - If some are pruned:
 - Initializes logits with a **very negative** constant (`LOGIT_FILL`).
 - Runs head only on kept candidates.
 - Fills logits/TTC outputs for kept candidates; pruned ones remain low-probability.
 - Denormalizes TTC and accumulates:
 - `logits_list, labels_list, pred_ttc_s_list, gt_ttc_s_list`.
- `_metrics_from_logits()` computes:
 - `accuracy`
 - `mAP`
 - `ttc_mae_seconds`
 - `num_candidates`

Metrics are saved as:

- `local_extraction/runs/Track_C/metrics/trackC_val_rateXX_<timestamp>.json`

with additional fields:

- `rgtp_enabled, rgtp_rate_request, rgtp_mean_fraction_pruned`.

Semantics: Track C does not change how **nouns / verbs / TTC** are defined; it only changes **which candidate tokens** are processed by the head. The evaluation is done on the same “is_positive + TTC” labels originating from Stage B manifests.

6. How Noun, Verb, and TTC Evolve Through the Pipeline

Putting it all together:

1. Ego4D STA annotations:

- Each object has (`verb_category_id, noun_category_id, time_to_contact, box`).

2. Label builder (`sta_builder_local.py`):

- Reads those fields and constructs `StaRec` entries.
- Optionally emits:

- YOLO labels where **noun IDs (or a single “next_active” class)** become detection class IDs.
- Head manifests where each decision frame’s **min-TTC object** carries (`verb_id`, `noun_id`, `ttc`) and `gt_box`.

3. Track A / Stage A:

- Uses YOLO to generate **noun-agnostic proposals** (`candidates.jsonl`).
- YOLO can be multi-class (noun-aware) or single-class; current thesis baseline follows the decision record and treats it as **generic proposals**, prioritizing **recall@K**.

4. Track A / Stage B:

- Aligns proposals with **head manifests**:
 - Attaches `verb_id`, `noun_id`, `ttc` from the min-TTC GT object per frame.
 - Computes `is_positive` via IoU with `gt_box`.
- Outputs `head_train.jsonl` / `head_val.jsonl` where each candidate is labeled with:
 - `is_positive` (class label),
 - `ttc` (scalar),
 - and carries `verb_id` / `noun_id` as metadata.

5. Track B dataset and model:

- Converts Stage B manifests into training samples:
 - `cls = 1` for positive candidates, `0` otherwise.
 - `ttc` as regression target (`ttc_norm` after normalization).
- The fusion head predicts:
 - `cls_logits` (candidate-level classification; currently binary).
 - `ttc` (normalized TTC; denormalized for MAE evaluation).
- Evaluation:
 - **Accuracy and mAP** for candidate classification.
 - **TTC MAE (seconds)** for timing.
- Noun and verb IDs are kept in Stage B manifests but are **not yet trained or evaluated** as separate heads in this code version; they are reserved for extensions where `num_classes` would correspond to noun or verb categories.

6. Track C pruning:

- Uses the same labels and TTC targets.
- Prunes token sets per candidate at inference and re-evaluates the same metrics.

7. Relation to Thesis Plan and Metrics (N, N+V, N+ δ)

`Thesis_main/thesis_plan_tracks_ABC_v2.md` and `Thesis_main/review.md` describe **idealized metrics**:

- **N mAP** — noun correctness.
- **N+V mAP** — joint noun+verb correctness.
- **N+ δ mAP** — noun plus TTC bin correctness.

In the current **local_extraction** implementation:

- The infrastructure **fully preserves noun / verb / TTC labels** in:
 - The label builder (**StaRec**, head manifests).
 - Stage B head manifests (**verb_id**, **noun_id**, **ttc** per candidate).
- The **trained head** is configured as:
 - A **binary classifier** ("this candidate is the min-TTC GT object or not").
 - A **TTC regressor**.
- Final reported metrics are:
 - **Classification accuracy** and **binary mAP** over the "next-active vs other" label.
 - **TTC MAE** in seconds.

These correspond most directly to:

- "N+ δ -style" evaluation at the **box level**, where:
 - We evaluate **which box** is predicted as next-active.
 - We assess **how close in time** the predicted interaction is (via TTC MAE).

To reach full **N / N+V / N+ δ mAP** as in the thesis plan, the next steps (not yet implemented in code here) would be:

- Extend Stage B manifests and Track B dataset to produce multi-class labels for:
 - Noun ID per candidate.
 - Verb ID per candidate.
 - Possibly joint (verb, noun) indices or multi-task heads.
- Expand **TrackBHead** to output:
 - Separate noun and verb logits (or a joint ($N \times V$) head), alongside TTC.
- Add evaluation code that checks **noun and verb correctness simultaneously** with boxes and TTC.

The current codebase therefore **implements the full pipeline for boxes + TTC**, with the **data plumbing in place for nouns and verbs**, but intentionally keeps the model head and metrics **binary** for this thesis iteration.

8. Summary

- **Noun, verb, and TTC** originate from Ego4D STA object annotations.
- **sta_builder_local.py** **extracts** these semantics, uses **noun IDs** to build YOLO labels, and emits **head manifests** with (**verb_id**, **noun_id**, **ttc**) and GT boxes.
- Track A / Stage A runs YOLO to produce **candidate boxes**; Stage B **aligns** them to head manifests and labels them with **is_positive** and **ttc**, preserving verb/noun metadata.
- Track B consumes these Stage B manifests, training a **fusion head** that predicts **candidate-level class (next-active) and TTC**, evaluated via **accuracy, mAP, and TTC MAE**.
- Track C reuses the same labels to assess **pruning-aware** variants of the same model, trading off FLOPs vs accuracy and TTC MAE.
- The code structure is intentionally **future-proofed**: noun and verb IDs flow through manifests and can be promoted to explicit prediction targets, enabling the full N / N+V / N+ δ metrics described in the thesis plan once model heads and evaluation scripts are extended.